C is a programming language that was developed in the early 1970s by Dennis Ritchie at Bell Labs. It is one of the oldest and most widely used programming languages in the world.

Key Points About C:

1. **General-Purpose Language**: C is designed to be a versatile language, meaning it can be used to create a wide variety of programs—from operating systems like Windows and Linux to simple programs like calculators.
2. **Middle-Level Language**: C is often referred to as a middle-level language because it gives programmers access to the computer's hardware (like memory and CPU) while still being easier to read and write than assembly language (the language that directly controls the computer's hardware).
3. **Efficiency**: Programs written in C are usually very fast and efficient, which is why it's often used for system software and applications that require high performance.
4. **Portable**: C programs can be compiled and run on different types of computers with little or no modification, which makes it a portable language.
5. **Foundation of Other Languages**: Many modern programming languages like C++, Java, and Python are influenced by C. Knowing C gives you a strong foundation for learning these other languages.

C is often considered a **middle-level language**, although it has characteristics of both **low-level** and **high-level** languages. Here's why:

Why C is Middle-Level:

1. **Low-Level Features**:
   - C provides **direct access to memory** and hardware through pointers, which is a feature common in low-level languages like Assembly.
   - You can manipulate individual bits and bytes, which gives you control over the system's hardware, like a low-level language.
   - It's used to write system software, such as operating systems and device drivers, which require direct interaction with hardware.
2. **High-Level Features**:
   - C also has high-level constructs like **functions**, **loops**, and **conditional statements** that make it easier to write, read, and understand, just like high-level languages such as Python or Java.
   - It supports modular programming, which allows you to break down a program into smaller, reusable components, a feature typical of high-level languages.

Real-Life Analogy:

Think of C as a **Swiss Army knife**. Just like this tool has various functions (both basic and complex), C gives you the flexibility to perform high-level tasks while also being able to control hardware in a detailed, low-level way when needed.

Summary:

- **Low-Level**: C can interact closely with the machine's hardware, giving you precise control.

- **High-Level**: C has abstractions that make it easier for humans to write programs without needing to worry about every single detail.

That's why C is called a **middle-level language**—it bridges the gap between low-level and high-level languages.

Real-Life Analogy:

Think of C as the "blueprint" language for building software. Just like an architect creates blueprints to construct buildings, a programmer uses C to create software that tells the computer exactly what to do.

Simple Example:

Here's a very simple C program:

```
#include <stdio.h>

int main() {
   printf("Hello, World!\n");
   return 0;
}
```

Writing a C program follows a specific syntax or structure that ensures the code can be compiled and run correctly. Here's a breakdown of the basic syntax with a simple example:

1. Header Files

- **#include**: Used to include standard libraries that provide useful functions and tools.
- Example: #include <stdio.h> includes the Standard Input Output library, which allows you to use functions like printf, scanf.

2. Main Function

- **int main()**: This is the entry point of every C program. The program starts executing from here.
- **Braces {}**: The code inside the braces is the body of the main function.

3. Statements

- **Statements** are the instructions you give to the computer, each ending with a semicolon ;.
- Example: printf("Hello, World!\n"); prints the message to the screen.

4. Return Statement

- **return 0;**: This ends the main function and returns 0 to indicate that the program has executed successfully.

5. Comments

- **Comments** are used to add notes or explanations within the code. They are ignored by the compiler.
- **Single-line comment**: // This is a comment
- **Multi-line comment**: /* This is a comment */

Basic Structure of a C Program:

```
#include <stdio.h>  // Include standard input/output library

// Main function where the program begins
int main() {
   // Print a message to the screen
   printf("Hello, World!\n");

   // End the program
   return 0;
}
```

Explanation of the Example:

1. **#include <stdio.h>**: This line includes the necessary library for input and output functions like printf.
2. **int main() { ... }**: This is the main function. The code inside the {} braces is what the computer will execute.
3. **printf("Hello, World!\n");**: This line prints the text "Hello, World!" to the screen. The \n is a newline character that moves the cursor to the next line after printing.
4. **return 0;**: This ends the program, returning 0 to indicate that everything worked correctly.

Key Points:

- **Case Sensitivity**: C is case-sensitive, meaning main and Main are different.
- **Semicolons**: Every statement must end with a semicolon.
- **Curly Braces**: Used to group multiple statements, defining the start and end of functions or control structures.

By following this structure, you can write a simple C program that the computer can understand and execute.

No, you should **not** use float main() instead of int main() in C. The **correct return type for the main function is int**, according to the C standard.

Here's why:

1. **C Standard Requirement**:
   - The C standard specifies that the main function must return an int value. This indicates that the program executed successfully or encountered an error.

- When you write int main(), the program returns an integer value, typically 0, which signals that the program completed successfully. A non-zero value can indicate an error.

2. **Return Value Expectation**:
   - The operating system expects main() to return an integer. If you return something else, like float, it can cause undefined behavior or errors, since the operating system won't get the expected result.
3. **float main()**:
   - If you use float main(), your program might still compile on some compilers, but it's non-standard and could cause issues on different systems. This is because the operating system expects main() to return an integer, not a floating-point number.

Correct Syntax:

```
#include <stdio.h>

int main() {
   printf("Hello, World!\n");
   return 0;  // Return type is int, indicating success
}
```

Why int and Not float?

- The return type int is simple and sufficient for signaling success or failure to the operating system.
- float is used for more complex numerical operations where decimal values are necessary, but for the main function, you don't need to return a floating-point number.

Summary:

- You **must** use int main() in C.
- Using float main() would break the standard and cause issues, as the operating system expects an integer return value.

Data Types in C

In C, **data types** specify the type of data a variable can store. Each data type also has a size (the amount of memory it consumes), which may vary slightly depending on the system (32-bit vs. 64-bit), but the general sizes are consistent on most platforms.

Here are the basic data types in C along with their definitions and typical memory usage:

1. int (Integer)

- **Definition**: Stores whole numbers (positive, negative, or zero) without any decimal places.
- **Memory Size**: Typically 4 bytes (32 bits).
- **Range**: -2,147,483,648 to 2,147,483,647 (on most 32-bit systems).

```
int number = 25;
```

2. float (Floating-Point)

- **Definition**: Stores real numbers with single precision, meaning it includes decimal points.
- **Memory Size**: Typically 4 bytes (32 bits).
- **Range**: Around 1.2E-38 to 3.4E+38.

```
float temperature = 36.5;
```

3. double (Double Precision Floating-Point)

- **Definition**: Stores real numbers with double precision (more accuracy than float).
- **Memory Size**: Typically 8 bytes (64 bits).
- **Range**: Around 2.3E-308 to 1.7E+308.

```
double pi = 3.1415926535;
```

4. char (Character)

- **Definition**: Stores a single character or ASCII value.
- **Memory Size**: 1 byte (8 bits).
- **Range**: 0 to 255 (for unsigned char), -128 to 127 (for signed char).

```
char letter = 'A';
```

5. short int or short

- **Definition**: A smaller version of the int type, used to save memory when you know the number will be small.
- **Memory Size**: Typically 2 bytes (16 bits).
- **Range**: -32,768 to 32,767.

```
short int smallNumber = 100;
```

6. long int or long

- **Definition**: A larger version of the int type, used when you need to store larger whole numbers.
- **Memory Size**: Typically 4 bytes (32 bits) or 8 bytes (64 bits) on 64-bit systems.
- **Range**: On 64-bit systems, -9,223,372,036,854,775,808 to 9,223,372,036,854,775,807.

```
long int bigNumber = 1000000;
```

7. long double

- **Definition**: An extended precision floating-point number, offering more precision than double.
- **Memory Size**: Typically 16 bytes (128 bits).
- **Range**: Varies but significantly higher precision and range than double.

```
long double preciseNumber = 2.718281828459;
```

8. unsigned int

- **Definition**: Stores only positive whole numbers, with no negative values, giving a larger range for positive numbers.
- **Memory Size**: Typically 4 bytes (32 bits).
- **Range**: 0 to 4,294,967,295 (for unsigned 32-bit integers).

```
unsigned int positiveNumber = 300;
```

Summary:

- **Integers (int, short, long)** store whole numbers.
- **Floating-point types (float, double, long double)** store real numbers with decimal points.
- **Character type (char)** stores single characters.

Naming Conventions

In C, naming conventions for variables are important for writing readable, maintainable, and error-free code. Here are some common **variable naming rules and best practices**:

1. Basic Rules for Naming Variables:

- **Only Letters, Digits, and Underscores**: Variable names can contain letters (A-Z or a-z), digits (0-9), and underscores (_), but no other symbols (like @, #, $, etc.).
- **Must Start with a Letter or Underscore**: A variable name cannot start with a digit. It must begin with a letter or an underscore. However, starting with an underscore is generally discouraged for non-system variables.
    - **Correct**: age, _count, numberOfItems
    - **Incorrect**: 123number, $total
- **No Spaces Allowed**: Variable names cannot contain spaces.
    - **Correct**: totalScore, highScore
    - **Incorrect**: total score, high score
- **Case Sensitive**: Variable names are case-sensitive, meaning age, Age, and AGE are considered different variables.
    - **Example**: totalAmount and TotalAmount are two distinct variables.
- **Reserved Keywords**: You cannot use C **keywords** (such as int, float, if, else, etc.) as variable names.
    - **Correct**: myNumber
    - **Incorrect**: int, float

2. Naming Conventions (Best Practices):

Following naming conventions helps make your code more readable and maintainable.

- **Use Descriptive Names**: Choose meaningful names that describe what the variable represents. This helps others (and your future self) understand the purpose of the variable.
  - **Good Example**: studentAge, totalPrice
  - **Bad Example**: x, y, a1, b2
- **Camel Case**: Start with a lowercase letter, and for multi-word names, capitalize the first letter of each word.
  - **Example**: studentAge, totalScore
- **Snake Case**: Use underscores to separate words. This is less common in C but sometimes used.
  - **Example**: student_age, total_score
- **Avoid Single-Letter Names**: Unless used in loops or for very short-term, simple variables (like i, j, k for loop counters), avoid single-letter names. They make the code harder to understand.
  - **Good Example**: totalMarks, averageHeight
  - **Bad Example**: t, h
- **Consistency**: Stick to a consistent naming style throughout your code (either camelCase or snake_case) to avoid confusion.
- **Avoid Starting with Underscore**: While it's legal to start a variable name with an underscore (_), it's generally discouraged for regular variables because underscore-prefixed names are often used for system or compiler-defined variables.
  - **Correct**: studentScore
  - **Avoid**: _studentScore

3. Example of Good Variable Names:

```
int studentAge;        // Describes the age of the student
float totalPrice;      // Describes the total price of an item or items
char firstInitial;     // Holds the first letter of a name
int numberOfItems;     // Describes the count of items
```

4. Examples of Bad Variable Names:

```
int a;           // Not descriptive
float 1total;    // Starts with a digit (invalid)
char first name;  // Contains space (invalid)
int $amount;     // Contains special symbol (invalid)
int float;       // Uses a reserved keyword (invalid)
```

5. Common Use of Short Variables in Special Cases:

Loop Variables: Short variable names like i, j, and k are often used in loops or for simple counters.

```
for (int i = 0; i < 10; i++) {
   // Do something
}
```

-

6. Constants Naming Convention:

Use **all uppercase letters** for constants, with underscores separating words, to clearly distinguish them from variables.

c

```
#define MAX_SIZE 100
const float PI = 3.14;
```

-

Summary:

- **Start** with a letter or underscore.
- **Be descriptive** to make the code easy to understand.
- **Use camelCase** or **snake_case** consistently.
- **Avoid keywords** and special characters.
- **Use constants** in all uppercase letters.

Tokens in C

1. Tokens in C

In C, **tokens** are the smallest elements that are meaningful to the compiler. C programs are composed of various tokens. These include:

- **Keywords**
- **Identifiers**
- **Constants**
- **Operators**
- **Special Symbols**

1.1 Keywords:

Keywords are reserved words in C that have special meanings defined by the language. You cannot use them as variable names or identifiers.

Examples: int, float, if, else, return

1.2 Identifiers:

Identifiers are names used to identify variables, functions, or any user-defined items. They must start with a letter or an underscore and can be followed by letters, digits, or underscores.

c

```
int age;
float totalPrice;
```

1.3 Constants:

Constants are values that cannot be modified during the execution of the program.

c

```
#define PI 3.14  // PI is a constant value
```

1.4 Operators:

Operators are symbols used to perform operations on variables and values.

```
int a = 5 + 3;  // + is an arithmetic operator
```

1.5 Special Symbols:

These include symbols like [], (), {}, ,, ;, #, and &, which have specific meanings in the language syntax.

---

2. Data Types in C

C has several data types used to define variables' types. Each data type specifies the type of data that the variablC Supports a rich set of built-in Operators. Operators are symbols that are used to perform some operation or a set of operations on a variable or a set of variables. C has a set of operators to perform specific mathematical and logical computations on operands.

Introduction to Operators in C Language

operators in C are that symbols which work on operands. Operator in C language is used to perform specific mathematical or logical computations on the operands and it reduces a single value.

Operators in C language, are classified into several categories.

Arithmetic Operators

Relational Operators

Shift Operators

Logical Operators

Bitwise Operators

Ternary or Conditional Operators

Assignment Operators

Misc Operators

Special Operators

Operators in C Language

Arithmetic Operators

An arithmetic operator is used to perform arithmetic/mathematical operations on operands. Some of the arithmetic operators are (+, -, *, /, %, ++, --)

Operator	Name of Operator	What it does	How it is used
+	Unary Plus	Add two Operands	a+b
-	Unary Minus	Subtracts the second operand from the first.	a-b
*	Multiplication	Multiplies both operands.	a*b
/	Division	Divides numerator by de-numerator.	a/b
%	Modulus	return remainder, after an integer division.	a%b
++	Increment Operator	increases the integer value by one.	a++
- -	Decrement Operator	decreases the integer value by one.	a- -

Relational Operators

Relational operators help in making a relationship or comparison between two operands with which they are used. Hence, relational operators help us make decisions in the program and their end result is either true or false. Some of the relation operators are (==, !=, <, >, <=, >=)

Example:

a == b

The expression given above, we used an equality operator that means it checks the value of a and b if both values are the same then it will return true otherwise it will return false.

Operator	Name of Operator	What it does	Return value
==	Equality Operator	checks if a == b	Boolean
!=	Not equal to	checks if a != b	Boolean
<	Less than	checks if a < b	Boolean
>	Greater than	checks if a > b	Boolean
<=	Less than or equal to	checks if a<=b	Boolean
>=	Greater than or equal to	checks if a>=b	Boolean

Shift Operators

The Shift Operators is used when we want to shift a binary bit either in the left direction or right direction.

Shift Operators are classified into two categories C Language:

Left Shift Operator: Left Shift Operator performs operations on the binary bits. The left shift operator is a type of binary operator so we need two operands to shift the position of the bits to the left side and add zeroes to the empty space on the right side after shifting the bits.

Syntax:

varName << no_of_position

The output of the left shift operator, will be equivalent to multiplying varName with 2 ^ no_of_position (2 raised to power no_of_position)

a << n = a * (2 ^ n)

Example:

Input: 25 << 2

Output: 100

Right Shift Operator: Right Shift Operator performs operations on the binary bits. The Right shift operator is a type of binary operator so we need two operands to shift the position of the bits to the right side and add zeroes to the empty space on the left side after shifting the bits.

Syntax:

varName >> no_of_position

The output of the right shift operator, will be equivalent to dividing varName with 2^no_of_position (2 raised to power no_of_position)

a >> n = a / (2 ^ n)

Example:

Input: 25 >> 2

Output: 6

Operator	Name of the Operator	What it does	How it is used
<<	Left Shift Operator	shifts the number of bits to the left side	a << 1
>>	Right Shift Operator	shifts the number of bits to the right side	a >> 2

Logical Operators

The logical operators are used when we want to check or test more than one condition and make decisions. Some of the logical operators are(&&, ||, !).

Example:

(a > b) && x == 100

The logical expression given above is true only if a > b is true and x == 100 is true. if either (or both) of them are false, the expression is false.

Operator Name of the operator What it does How it is used/output

&& logical AND returns true if both side operands value is true otherwise returns false Boolean

|| logical OR returns true if one of the operand's value is true or both of the operand's values is true otherwise returns false Boolean

! logical Not returns true if the condition in consideration is not satisfied Otherwise returns false Boolean

Bitwise Operators

An Bitwise operator is used for the manipulation of data at the bit level. These operators are not applied for the float and double datatype.

Bitwise operator first converts the integer into its binary representation then performs its operation. Bitwise operators subsist of two digits, either 0 or 1. Some of the bitwise operators are (&, | , ^, ~)

Note: Shift Bitwise operators are used to shift the bits right to left. Some of the shift bitwise operators are(<<, >>)

We use the following truth table for the Bitwise Operators:

A	B	A & B (Bitwise AND)	A | B (Bitwise OR)	A ^ B (Bitwise XoR)
1	1	1	1	0
0	1	0	1	1
1	0	0	1	1
0	0	0	0	0

Example:

a = 5, b = 6

a & b = 4 (In Decimal) a | b = 7 (In Decimal) a ^ b = 3 (In Decimal)

In the above example, We have two variables a and b.

a = 5 (In Decimal);

b = 6 (In Decimal);

So, a's binary representation is 0101(5) and b's binary representation is 0110(6)

AND Operation	OR Operation	XOR Operation
0101	0101	0101
& 0110	| 0110	^ 0110
-----	----	----
0100 = 4	0111 = 7	0011 = 3

Operator	Name of Operator	What it does	How it is used
&	bitwise AND	bitwise AND operator do AND of every corresponding bits of both operands and output 1 (true) if both operands have 1 at that position otherwise 0(false).	a & b
|	bitwise OR	bitwise OR operator do OR operation of every corresponding bits of both operands and output 0 (false) if both operands have 0 at that position otherwise 1(true).	a | b
~	bitwise complement	performs complement operation on an operand and bitwise complement changes 1 to 0 and 0 to 1	~a
^	bitwise exclusive OR	returns 1 if the corresponding bits of two operands are opposite else 0	a^b
<<	shift left	shifts the number of bits to the left side	a << 1
>>	shift right	shifts the number of bits to the right side	a >> 1

Ternary or Conditional Operators

the ternary or conditional operators is used to construct the conditional expression. A conditional operator pair "?:"

Syntax:

exp1 ? exp2 : exp3

Here exp1, exp2, exp3 are expressions.

The Operator ?: works as follows: exp1 is evaluated first. If it is true, then the expression exp2 is evaluated and becomes the value of the expression. If exp1 is false, then exp3 is evaluated and its value becomes the value of the expression.

Example:

```
a = 100;

b = 200;

x = (a > b) ? a : b;
```

In the above example, we have two variables a and b. x, will be assigned the value of b because a>b is false.

Misc Operators

Misc Operator is type of Operator in C language. It is also called Miscellaneous Operator. Some of the Misc operators are (sizeof() ?: , & *)

Example:

Input: a=(1, 2, 3)

Output: 3

Operator	Name of Operator	What it does	How it is used
sizeof()	sizeof	It returns the size of variable	if variable a is a interger variable the sizeof(a) will return 4
?:	conditional or ternary operator	if the condition is true then it returns the value of x else value of y	condition?x
cast	type cast	it converts one datatype to another datatype	int(5.260) would return 5
,	comma operator	Used to link the related expressions together	a = (1,2,3) would return 3
&	Address Operator	returns the address of the variable.	&a
*	pointer operator	pointer to a variable	*a

Assignment Operators

An assignment operator is used to assign values to the operands. Some of the assignment operators are (=, +=, -=, *=, /=, %=)

Eample:

a = b

In the above example, we are assigning b's value to variable a.

Operator	Name of Operator	What it does	How it is used
=	assignment	assign value of variable b to variable a	a = b
+=	plus assign	a = a+b (adds values of a to b and assign this value to a)	a += b
-=	minus assign	a = a-b (subtracts values of b from a and assign this value to a)	a -= b
*=	times assign	a = a*b (Multiplies a with b and assign the value to a)	a *= b
/=	div assign	a = a/b (divides a by b and assigns the value to a)	a /= b
%=	Mod assign	a = a%b (divides a by b and assigns the value of the remainder to a)	a %= b

Special Operators

C supports some special operators some of the special operators are (comma operator, address operator, size of operator, pointer operator)

Example:

m = sizeof(a)

Operator	Name of Operator	What it does	How it is used
,	Comma	Used to link the related expressions together	value = (x=10, y=5)
&	Address Operator	returns the address of the variable.	&a
sizeof()	sizeof	returns the size of a variable	m = sizeof(a)

Let's Understand the sizeof() operator with the help of program

```
#include <stdio.h>

int main() {
    int a = 100;
    printf("%d", (int)sizeof(a));

    return 0;
}
```

Output:

4

Note: sizeof() may give different output according to the compiler.

Precedence(or priority) and Associativity of Operators in C

Precedence determines which operator is performed first in an expression if there are more than one operator of different precedence(lower precedence means higher priority). Associativity determines in which direction we should start computing the operators having the same precedence. The table shows the precedence and their associativity among operators.

Token	Operator	Precedence	Associativity
()	function call	1	left-to-right
[]	array element	1	left-to-right
++	postfix increament	1	left-to-right
- -	postfix decreament	1	left-to-right
++	prefix increament	2	right-to-left
- -	prefix decreament	2	right-to-left
+	unary plus	2	right-to-left
-	unary minus	2	right-to-left
!	Logical negation	2	right-to-left
~	one's complement	2	right-to-left
*	indirection	2	right-to-left
&	address 2	right-to-left	
sizeof	size(in bytes)	2	right-to-left
(type)	type cast	2	right-to-left
*	multiplication	3	left-to-right
/	division3	left-to-right	
%	modulus	3	left-to-right
+	addition	4	left-to-right
-	subtraction	4	left-to-right

<<	left shift	5	left-to-right
>>	right shift	5	left-to-right
<	less than	6	left-to-right
<=	less than or equal to	6	left-to-right
>	greater than	6	left-to-right
>=	greater than or equal to	6	left-to-right
==	equality	7	left-to-right
!=	inequality	7	left-to-right
&	bitwise AND	8	left-to-right
^	bitwise XOR	9	left-to-right
|	bitwise OR	10	left-to-right
&&	Logical AND	11	left-to-right
||	Logical OR	12	left-to-right
?:	conditional expression	13	right-to-left
= *= /= %= += -= &= ^= |= <<= >>=	assignment operators	14	right-to-left
,	comma operator	15	left-to-right

Let's understand the precedence(or priority) operator with the help of programming example.

Example 1:

```
#include <stdio.h>

int main() {
   // operator priority
   int a = 5;
   a = 10,20,30;
```

```
    printf("%d", a);


    return 0;
}
```

Output:

10

explanation: priority or precedence for the values assigned to any variable is given from left to right.

Example 2:

```
#include <stdio.h>

int main() {
    // operator priority
    int x = 5;
    x = (10,20,30);
    printf("%d", x);


    return 0;
}
```

Output:

explanation: priority or precendence for the values indside brackets () assigned to any variable is given from right to left.

Conclusion

We perform operations on variables or operands by using operators in C

An expression is the combination of operands and operators that reduces a single value.

Arithmetic operators in C are used for mathmatical operations and include (+, -, *, /, %, ++, --).

Relational operators in C are used to compare 2 variables and include (==, !=, <, >, <=, >=).

Shift operators in C are used to shift bits of a number and include (<<, >>).

To check more than one condition we use logical operators and they include (&&, ||, !).

Bitwise operator are used to manipulate bits of a number and include (&, | , ^, ~).

Ternary operator (?) are used as a short-hand to write if-else condition.

Miscellaneous operator include (sizeof() ?: , & *).

To assign values we use assignment operators in C and they include (=, +=, -=, *=, /=, %=).e can hold.

2.1 Basic Data Types:

int: Used to store integers (whole numbers).
c

```
int age = 30;
```

-

float: Used to store floating-point numbers (numbers with decimal points).
c

```
float temperature = 36.5;
```

-

char: Used to store a single character.
c

```
char letter = 'A';
```

-

double: Used to store double-precision floating-point numbers.
c

```
double pi = 3.14159;
```

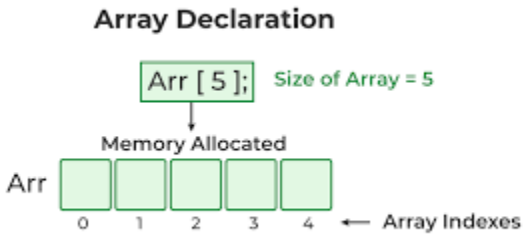
-

2.2 Derived Data Types:

a) **Arrays**: Collection of variables of the same type.

```
int numbers[5] = {1, 2, 3, 4, 5};
// Declare an array of four integers:
int myNumbers[4];

// Add elements
myNumbers[0] = 25;
myNumbers[1] = 50;
myNumbers[2] = 75;
myNumbers[3] = 100
```



b) Pointers in C

A **pointer** in C is a variable that stores the **memory address** of another variable. Pointers allow for more direct memory management and manipulation.

Basic Concepts:

- **Pointer Declaration**: A pointer is declared using the * symbol before the variable name.
- **Address-of Operator (&)**: This operator returns the memory address of a variable.
- **Dereference Operator (*)**: This operator accesses the value stored at the memory address the pointer is pointing to.

Pointer Syntax:

- int *ptr; // Declares a pointer to an integer

Example:

```
#include <stdio.h>
int main() {
    int num = 10;      // Regular variable
    int *ptr = &num;   // Pointer to the variable 'num'

    // Printing the value and address
    printf("Value of num: %d\n", num);
 // Outputs: 10
    printf("Address of num: %p\n", &num);
// Outputs: Address of num
    printf("Pointer ptr holds the address: %p\n", ptr);
// Outputs: Same address as &num
    printf("Value at the address held by ptr: %d\n", *ptr);
// Outputs: 10 (Dereferencing)

    return 0;
}
```

Explanation:

- int *ptr: Declares a pointer variable ptr that holds the memory address of an integer.
- ptr = &num: The pointer ptr is assigned the address of num using the & operator.
- *ptr: This dereferences the pointer, meaning it accesses the value stored at the memory address ptr is pointing to.

Why Use Pointers?:

- **Memory Efficiency**: Pointers allow you to pass large data structures (like arrays) to functions without copying the data.
- **Dynamic Memory Allocation**: You can use pointers to dynamically allocate memory at runtime using functions like malloc() and free().
- **Function Pointers**: Pointers can point to functions, allowing dynamic function calls.

2.3 User-Defined Data Types:

struct: Grouping of different data types.

```
struct student {
   int id;
   char name[50];
};
```

-

---

3. Operators in C

Operators in C are used to perform operations on variables and values.

3.1 Arithmetic Operators:

Used for mathematical calculations.

```
+ - * / %  // Addition, subtraction, multiplication, division, modulus
```

Example:

```
int sum = 5 + 3;  // sum will be 8
```

3.2 Relational Operators:

Used to compare values.

c

```
== != > < >= <=  // Equality, inequality, greater, less, etc.
```

Example:

c

```
if (a == b) { /* code */ }  // Checks if a is equal to b
```

3.3 Logical Operators:

Used for logical operations.

c

```
&&  ||  !  // AND, OR, NOT
```

Example:

c

```
if (a > 0 && b > 0) { /* code */ }  // True if both conditions are true
```

3.4 Assignment Operators:

Used to assign values to variables.

c

```
=  +=  -=  *=  /=  // Basic and compound assignment
```

Example:

c

```
a += 5;  // a = a + 5
```

3.5 Increment/Decrement Operators:

Used to increase or decrease a value by 1.

c

```
++  --  // Increment, decrement
```

Example:

c

```
a++;  // Increases a by 1
```

4. Flow of Control

Flow of control statements control the order in which the program executes statements. They include **conditional statements** and **loops**.

4.1 Conditional Statements

4.1.1 if-else:

The if-else statement allows you to execute code blocks conditionally based on a Boolean condition.

Syntax:

c

```
if (condition) {
   // Code to execute if condition is true
} else {
   // Code to execute if condition is false
}
```

Example:

```
int age = 18;
if (age >= 18) {
   printf("You are an adult.");
} else {
   printf("You are a minor.");
}
```

4.1.2 switch-case:

The switch-case construct is used to select one of many code blocks to execute based on the value of an expression.

Syntax:

```
switch (expression) {
   case value1:
      // Code to execute if expression == value1
      break;
   case value2:
      // Code to execute if expression == value2
      break;
   default:
      // Code to execute if no cases match
}
```

Example:

c

```c
int day = 2;
switch (day) {
   case 1:
      printf("Monday");
      break;
   case 2:
      printf("Tuesday");
      break;
   default:
      printf("Unknown day");
}
```

4.2 Loops

Loops are used to execute a block of code repeatedly as long as a condition is true.

4.2.1 while Loop:

The while loop checks the condition first, and if true, executes the code inside the loop.

Syntax:

c

```c
while (condition) {
   // Code to execute as long as condition is true
}
```

Example:

c

```c
int i = 0;
while (i < 5) {
   printf("%d\n", i);
   i++;
}
```

4.2.2 do-while Loop:

The do-while loop executes the code block once before checking the condition.

Syntax:

c

```
do {
   // Code to execute
} while (condition);
```

Example:

c

```
int i = 0;
do {
   printf("%d\n", i);
   i++;
} while (i < 5);
```

4.2.3 for Loop:

The for loop allows you to specify the initialization, condition, and increment/decrement in one line.

Syntax:

c

```
for (initialization; condition; increment) {
   // Code to execute as long as condition is true
}
```

Example:

c

```
for (int i = 0; i < 5; i++) {
   printf("%d\n", i);
}
```

Summary:

- **Tokens**: Smallest units in C like keywords, identifiers, constants, and operators.
- **Data Types**: Define the type and size of data, such as int, float, char, etc.
- **Operators**: Perform operations like arithmetic, comparison, and logical checks.
- **Flow of Control**: Includes conditional statements (if-else, switch-case) and loops (while, do-while, for), which control the execution flow in programs.

Arithmetic Operators

We are most familiar with the arithmetic operators. These operators are used to perform arithmetic operations on operands. The most common arithmetic operators are addition (+), subtraction (-), multiplication (*), and division (/).

In addition, the modulo (%) is an important arithmetic operator that computes the remainder of a division operation. Arithmetic operators are used in forming an arithmetic expression. These operators are binary in nature in the sense they need two operands, and they operate on numeric operands, which may be numeric literals, variables or expressions.

For example, take a look at this simple expression −

```
a + b
```

Here "+" is an arithmetic operator. We shall learn more about arithmetic operators in C in a subsequent chapter.

The following table shows all the arithmetic operators supported by the C language. Assume variable **A** holds 10 and variable **B** holds 20 then −

Show Examples

Operator	Description	Example
+	Adds two operands.	A + B = 30
−	Subtracts second operand from the first.	A − B = -10
*	Multiplies both operands.	A * B = 200
/	Divides numerator by de-numerator.	B / A = 2
%	Modulus Operator and remainder of after an integer division.	B % A = 0
++	Increment operator increases the integer value by one.	A++ = 11
--	Decrement operator decreases the integer value by one.	A-- = 9



Relational Operators

We are also acquainted with relational operators while learning secondary mathematics. These operators are used to compare two operands and return a boolean value (true or false). They are used in a boolean expression.

The most common relational operators are less than (<), greater than (>), less than or equal to (<=), greater than or equal to (>=), equal to (==), and not equal to (!=). Relational operators are also binary operators, needing two numeric operands.

For example, in the Boolean expression −

```
a > b
```

Here, ">" is a relational operator.

We shall learn more about with relational operators and their usage in one of the following chapters.

Show Examples

Operator	Description	Example
==	Checks if the values of two operands are equal or not. If yes, then the condition becomes true.	(A == B) is not true.
!=	Checks if the values of two operands are equal or not. If the values are not equal, then the condition becomes true.	(A != B) is true.
>	Checks if the value of left operand is greater than the value of right operand. If yes, then the condition becomes true.	(A > B) is not true.
<	Checks if the value of left operand is less than the value of right operand. If yes, then the condition becomes true.	(A < B) is true.
>=	Checks if the value of left operand is greater than or equal to the value of right operand. If yes, then the condition becomes true.	(A >= B) is not true.
<=	Checks if the value of left operand is less than or equal to the value of right operand. If yes, then the condition becomes true.	(A <= B) is true.

Explore our **latest online courses** and learn new skills at your own pace. Enroll and become a certified expert to boost your career.

Logical Operators

These operators are used to combine two or more boolean expressions. We can form a compound Boolean expression by combining Boolean expression with these operators. An example of logical operator is as follows −

```
a >= 50 && b >= 50
```

The most common logical operators are AND (&&), OR(||), and NOT (!). Logical operators are also binary operators.

Show Examples

Operator	Description	Example
&&	Called Logical AND operator. If both the operands are non-zero, then the condition becomes true.	(A && B) is false.
||	Called Logical OR Operator. If any of the two operands is non-zero, then the condition becomes true.	(A || B) is true.
!	Called Logical NOT Operator. It is used to reverse the logical state of its operand. If a condition is true, then Logical NOT operator will make it false.	!(A && B) is true.

We will discuss more about Logical Operators in C in a subsequent chapter.



Bitwise Operators

Bitwise operators let you manipulate data stored in computer's memory. These operators are used to perform bit-level operations on operands.

The most common bitwise operators are AND (&), OR (|), XOR (^), NOT (~), left shift (<<), and right shift (>>). Here the "~" operator is a unary operator, while most of the other bitwise operators are binary in narure.

Bitwise operator works on bits and perform bit−by−bit operation. The truth tables for &, "|", and "^" are as follows −

p	q	p & q	p | q	p ^ q
0	0	0	0	0

0	1	0	1	1
1	1	1	1	0
1	0	0	1	1

Assume A = 60 and B = 13 in binary format, they will be as follows −

A = 0011 1100

B = 0000 1101

A&B = 0000 1100

A|B = 0011 1101

A^B = 0011 0001

~A = 1100 0011

The following table lists the bitwise operators supported by C. Assume variable 'A' holds 60 and variable 'B' holds 13, then −

Show Examples

Operator	Description	Example
&	Binary AND Operator copies a bit to the result if it exists in both operands.	(A & B) = 12, i.e., 0000 1100
|	Binary OR Operator copies a bit if it exists in either operand.	(A | B) = 61, i.e., 0011 1101
^	Binary XOR Operator copies the bit if it is set in one operand but not both.	(A ^ B) = 49, i.e., 0011 0001
~	Binary One's Complement Operator is unary and has the effect of 'flipping' bits.	(~A) = ~(60), i.e,. -0111101
<<	Binary Left Shift Operator. The left operands value is moved left by the number of bits specified by the right operand.	A << 2 = 240 i.e., 1111 0000
>>	Binary Right Shift Operator. The left operands value is moved right by the number of bits specified by the right operand.	A >> 2 = 15 i.e., 0000 1111

Assignment Operators

As the name suggests, an assignment operator "assigns" or sets a value to a named variable in C. These operators are used to assign values to variables. The "=" symbol is defined as assignment operator in C, however it is not to be confused with its usage in mathematics.

The following table lists the assignment operators supported by the C language −

Show Examples

Operator	Description	Example
=	Simple assignment operator. Assigns values from right side operands to left side operand	C = A + B will assign the value of A + B to C
+=	Add AND assignment operator. It adds the right operand to the left operand and assign the result to the left operand.	C += A is equivalent to C = C + A
-=	Subtract AND assignment operator. It subtracts the right operand from the left operand and assigns the result to the left operand.	C -= A is equivalent to C = C - A
*=	Multiply AND assignment operator. It multiplies the right operand with the left operand and assigns the result to the left operand.	C *= A is equivalent to C = C * A
/=	Divide AND assignment operator. It divides the left operand with the right operand and assigns the result to the left operand.	C /= A is equivalent to C = C / A
%=	Modulus AND assignment operator. It takes modulus using two operands and assigns the result to the left operand.	C %= A is equivalent to C = C % A
<<=	Left shift AND assignment operator.	C <<= 2 is same as C = C << 2
>>=	Right shift AND assignment operator.	C >>= 2 is same as C = C >> 2
&=	Bitwise AND assignment operator.	C &= 2 is same as C = C & 2
^=	Bitwise exclusive OR and assignment operator.	C ^= 2 is same as C = C ^ 2

|=	Bitwise inclusive OR and assignment operator.	C |= 2 is same as C = C | 2

Hence, the expression "a = 5" assigns 5 to the variable "a", but "5 = a" is an invalid expression in C.

The "=" operator, combined with the other arithmetic, relational and bitwise operators form augmented assignment operators. For example, the += operator is used as add and assign operator. The most common assignment operators are =, +=, -=, *=, /=, %=, &=, |=, and ^=.



Misc Operators ↦ sizeof & ternary

Besides the operators discussed above, there are a few other important operators including **sizeof** and **? :** supported by the C Language.

Show Examples

Operator	Description	Example
sizeof()	Returns the size of a variable.	sizeof(a), where a is integer, will return 4.
&	Returns the address of a variable.	&a; returns the actual address of the variable.
*	Pointer to a variable.	*a;
? :	Conditional Expression.	If Condition is true ? then value X : otherwise value Y

Operators Precedence in C

Operator precedence determines the grouping of terms in an expression and decides how an expression is evaluated. Certain operators have higher precedence than others; for example, the multiplication operator has a higher precedence than the addition operator.

For example, x = 7 + 3 * 2; here, x is assigned 13, not 20 because operator * has a higher precedence than +, so it first gets multiplied with 3*2 and then adds into 7.

Here, operators with the highest precedence appear at the top of the table, those with the lowest appear at the bottom. Within an expression, higher precedence operators will be evaluated first.

Show Examples

Category	Operator	Associativity

Postfix	() [] -> . ++ - -	Left to right
Unary	+ - ! ~ ++ - - (type)* & sizeof	Right to left
Multiplicative	* / %	Left to right
Additive	+ -	Left to right
Shift	<< >>	Left to right
Relational	< <= > >=	Left to right
Equality	== !=	Left to right
Bitwise AND	&	Left to right
Bitwise XOR	^	Left to right
Bitwise OR	|	Left to right
Logical AND	&&	Left to right
Logical OR	||	Left to right
Conditional	?:	Right to left
Assignment	= += -= *= /= %=>>= <<= &= ^= |=	Right to left
Comma	,	Left to right

Other Operators in C

Apart from the above, there are a few other operators in C that are not classified into any of the above categories. For example, the increment and decrement operators (++ and --) are unary in nature and can appear as a prefix or postfix to the operand.

The operators that work with the address of memory location such as the address-of operator (&) and the dereference operator (*). The sizeof operator (sizeof) appears to be a keyword but really an operator.

C also has the type cast operator (()) that forces the type of an operand to be changed. C also uses the dot (.) and the arrow (->) symbols as operators when dealing with derived data types such as struct and union.

The C99 version of C introduced a few additional operators such as auto, decltype.

A single expression in C may have multiple operators of different type. The C compiler evaluates its value based on the operator precedence and associativity of operators. For example, in the following expression −

```
a + b * c
```

The multiplication operand takes precedence over the addition operator.

Conditional or Ternary Operator (?:) in C

The **conditional operator in C** is kind of similar to the if-else statement as it follows the same algorithm as of if-else statement but the conditional operator takes less space and helps to write the if-else statements in the shortest way possible. It is also known as the **ternary operator in C** as it operates on three operands.

Syntax of Conditional/Ternary Operator in C

The conditional operator can be in the form

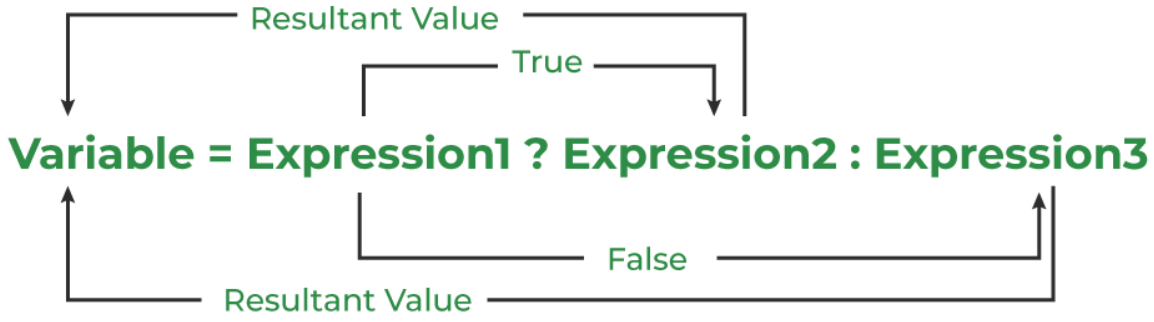
variable = Expression1 **?** *Expression2* **:** *Expression3;*

Or the syntax can also be in this form

variable = (condition) **?** *Expression2* **:** *Expression3*;

Or syntax can also be in this form

(condition) **?** *(variable = Expression2)* **:** *(variable = Expression3)*;



It can be visualized into an if-else statement as:

```
if(Expression1)
{
  variable = Expression2;
```

```
}
else
{
  variable = Expression3;
}
```

Since the Conditional Operator '?:' takes three operands to work, hence they are also called **ternary operators**.

Note: The ternary operator have third most lowest precedence, so we need to use the expressions such that we can avoid errors due to improper operator precedence management.

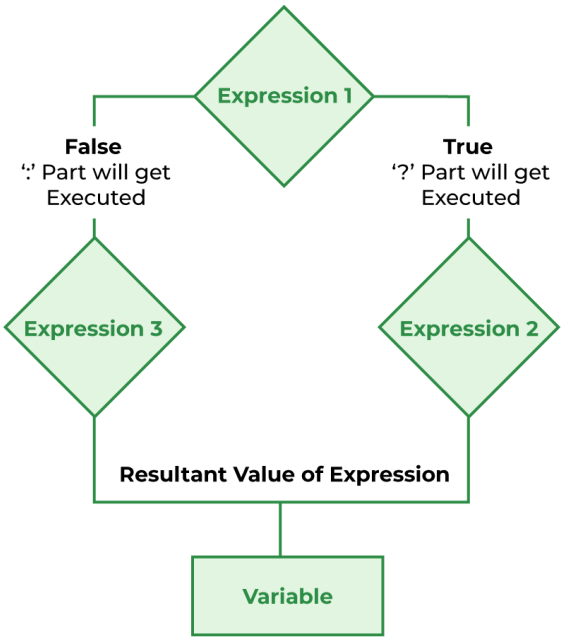
Working of Conditional/Ternary Operator in C

The working of the conditional operator in C is as follows:

- **Step 1: Expression1** is the condition to be evaluated.
- **Step 2A:** If the condition(**Expression1**) is True then **Expression2** will be executed.
- **Step 2B**: If the condition(**Expression1**) is false then **Expression3** will be executed.
- **Step 3:** Results will be returned.

Flowchart of Conditional/Ternary Operator in C

To understand the working better, we can analyze the flowchart of the conditional operator given below.



Flowchart of conditional/ternary operator in C

Examples of C Ternary Operator

Example 1: C Program to Store the greatest of the two Numbers using the ternary operator

C

```
// C program to find largest among two
// numbers using ternary operator

#include <stdio.h>

int main()
{
   int m = 5, n = 4;

   (m > n) ? printf("m is greater than n that is %d > %d",
                m, n)
           : printf("n is greater than m that is %d > %d",
                n, m);

   return 0;
}
```

Output

m is greater than n that is 5 > 4

Conditional Statements in C

Conditional statements in C allow the program to make decisions based on certain conditions. These statements control the flow of execution depending on whether a condition evaluates to `true` (non-zero) or `false` (zero). The most commonly used conditional statements in C are:

1. **if statement**
2. **if-else statement**
3. **else-if ladder**
4. **nested if**
5. **switch-case statement**

1. if Statement

The `if` statement executes a block of code **only if a specified condition is true**.

Syntax:

```
if (condition) {
    // code to execute if condition is true
}
```

Example:

```
#include <stdio.h>

int main() {
    int number = 10;

    if (number > 5) {
        printf("The number is greater than 5.\n");
    }
```

```
    return 0;
}
```

Explanation: If `number > 5` is true, the statement inside the `if` block is executed.

Output:

csharp

```
The number is greater than 5.
```

2. if-else Statement

The `if-else` statement provides an alternative block of code that executes if the condition in the `if` statement is **false**.

Syntax:

```
if (condition) {
    // code to execute if condition is true
} else {
    // code to execute if condition is false
}
```

Example:

```
#include <stdio.h>

int main() {
    int number = 3;

    if (number > 5) {
        printf("The number is greater than 5.\n");
```

```
    } else {
        printf("The number is not greater than 5.\n");
    }

    return 0;
}
```

Output:

```
The number is not greater than 5.
```

3. else-if Ladder

The `else-if` ladder is used when there are multiple conditions to check. It evaluates conditions one by one from top to bottom, and the block of the first `true` condition is executed. If none of the conditions are true, the `else` block is executed.

Syntax:

c

```
if (condition1) {
    // code to execute if condition1 is true
} else if (condition2) {
    // code to execute if condition2 is true
} else {
    // code to execute if none of the above conditions are true
}
```

Example:

c

```
#include <stdio.h>

int main() {
    int number = 0;

    if (number > 0) {
        printf("The number is positive.\n");
    } else if (number < 0) {
        printf("The number is negative.\n");
    } else {
        printf("The number is zero.\n");
    }

    return 0;
}
```

Output:

```
The number is zero.
```

4. Nested if Statement

The `if` statements can be nested, meaning you can have an `if` statement inside another `if` or `else` block.

Syntax:

```
if (condition1) {
    if (condition2) {
        // code to execute if both condition1 and condition2 are true
    }
```

```
}
```

Example:

```
#include <stdio.h>

int main() {
    int num = 20;

    if (num > 10) {
        if (num < 30) {
            printf("The number is between 10 and 30.\n");
        }
    }

    return 0;
}
```

Output:

csharp

```
The number is between 10 and 30.
```

5. switch-case Statement

The `switch-case` statement is used to simplify a series of `if-else-if` statements when you are comparing a single variable with different values. It is generally more readable and efficient for such use cases.

Syntax:

```
switch (variable) {
    case constant1:
        // code to execute if variable == constant1
        break;
    case constant2:
        // code to execute if variable == constant2
        break;
    default:
        // code to execute if none of the cases match
}
```

Example:

```
#include <stdio.h>

int main() {
    int day = 3;

    switch (day) {
        case 1:
            printf("Monday\n");
            break;
        case 2:
            printf("Tuesday\n");
            break;
        case 3:
            printf("Wednesday\n");
            break;
```

```
        default:
            printf("Invalid day\n");
    }

    return 0;
}
```

Output:

mathematica

```
Wednesday
```

Flow of Control with Conditional Statements

1. if:

- **Condition**: The expression inside the parentheses is evaluated.
- **True**: If the condition evaluates to true (non-zero), the block inside the `if` statement is executed.
- **False**: If the condition evaluates to false (zero), the block is skipped.

2. if-else:

- **Condition**: The expression inside the `if` is evaluated.
- **True**: If the condition is true, the `if` block is executed.
- **False**: If the condition is false, the `else` block is executed.

3. else-if Ladder:

- **Condition1**: The expression inside the first `if` is evaluated.
- **Condition2**: If the first condition is false, the next condition (`else if`) is evaluated, and so on.
- **Else**: If none of the conditions are true, the `else` block is executed.

4. switch-case:

- The `switch` expression is evaluated.
- The program compares the result with each case value.
- The matching case block is executed.
- The `break` statement prevents the execution of the next case block.

Conclusion:

Conditional statements are essential for controlling the flow of the program based on different conditions. They allow the program to take different actions based on user input, logical conditions, or the state of the system. Understanding how to use `if`, `if-else`, `else-if`, and `switch-case` statements is key to building decision-making programs in C.

Here is a comparison of the different **conditional statements in C**, highlighting when to use each, and their pros and cons.

1. if Statement

When to Use:

- Use the `if` statement when you need to execute code based on a single condition being **true**.

Pros:

- Simple and straightforward for checking one condition.
- Easy to read when the condition is basic.

Cons:

- Becomes harder to manage with complex conditions or multiple checks.

Example:

```
if (a > 10) {
    // Do something if a is greater than 10
}
```

2. if-else Statement

When to Use:

- Use the `if-else` statement when you want to take one action if the condition is **true**, and a different action if the condition is **false**.

Pros:

- Provides an alternative (else) when the `if` condition is false.

Cons:

- Can become hard to read if there are many nested `if-else` statements.

Example:

```
if (a > 10) {
    // Do something if a is greater than 10
} else {
    // Do something else if a is not greater than 10
}
```

3. else-if Ladder

When to Use:

- Use `else-if` when there are **multiple conditions** to check, and only one of them should be true.
- It's essentially a series of `if` statements, but only one branch can be executed.

Pros:

- Useful for checking multiple conditions in sequence.
- Only one block will be executed, even if multiple conditions are true.

Cons:

- The more conditions you add, the harder it is to read and maintain.
- Evaluates conditions in sequence, so it can be slower if conditions that are more likely to be true are checked later.

Example:

```
if (a > 10) {
    // Do something if a is greater than 10
} else if (a == 10) {
    // Do something else if a is equal to 10
} else {
    // Do something else if none of the above conditions are true
}
```

4. Nested if Statement

When to Use:

- Use **nested if** when you need to check a condition **inside** another condition.

Pros:

- Useful when you need to check a secondary condition only if the first condition is true.

Cons:

- Can become very hard to read if you nest too many `if` statements.
- Can result in deep nesting which makes debugging and maintaining the code harder.

Example:

```
if (a > 10) {
    if (b < 5) {
        // Do something if a is greater than 10 and b is less than 5
    }
}
```

5. switch-case Statement

When to Use:

- Use `switch-case` when you are checking a single variable or expression against **multiple possible values**.
- Useful when each condition compares the same variable to different values.

Pros:

- More readable and efficient than multiple `else-if` statements when checking for specific values of a single variable.
- Fast, as the compiler may optimize `switch-case` into a lookup table for constant values.

Cons:

- Limited to discrete values (such as integers or characters). Cannot be used for ranges or complex conditions.
- Must handle each case manually, which can lead to longer code if there are many cases.

Example:

```
switch (a) {
    case 1:
        // Do something if a is 1
        break;
    case 2:
        // Do something if a is 2
        break;
    default:
        // Do something if a is neither 1 nor 2
}
```

Comparison Table:

Feature	if	if-else	else-if Ladder	Nested if	switch-case
Use Case	Single condition	One condition, two outcomes	Multiple conditions in sequence	Conditions inside conditions	Multiple values for one variable
Complexity	Simple for one condition	Slightly more complex	More complex with each condition	Can get very complex	Simpler for checking values
Readability	Very readable for simple conditions	Readable, but can get long	Harder to read with many conditions	Hard to read if deeply nested	Highly readable
Efficiency	Efficient for simple conditions	Efficient	Becomes less efficient with many checks	Can slow down if deeply nested	Very efficient for discrete values

Usage with Ranges	Yes	Yes	Yes	Yes	No
Usage with Discrete Values	Yes	Yes	Yes	Yes	Best suited for discrete values
Real-life Example	Checking if a number is positive	Deciding between two options	Grading system (A, B, C)	Validating nested conditions	Menu selection in a program

Key Differences:

- **`if` vs `switch-case`**:
  - `if` can handle any kind of condition (ranges, complex comparisons), whereas `switch-case` is limited to discrete values (like integers or characters).
  - `switch-case` is often more efficient when dealing with a single variable compared to multiple `if-else` statements, as it can be optimized by the compiler.
- **`else-if Ladder` vs `switch-case`**:
  - An `else-if` ladder is useful when you have multiple ranges or conditions to check.
  - `switch-case` is better when you are checking a single variable against different exact values.
- **Nested if**:
  - Good for dependent conditions, but quickly becomes unreadable if overused.

Loops in C: `while` and `do-while` Loops

Loops allow you to repeatedly execute a block of code as long as a condition remains true. In C, two common types of loops are the **`while` loop** and the **`do-while` loop**. Let's explore their syntax, use cases, and differences.

---

1. `while` Loop

The `while` loop repeatedly executes a block of code **as long as the specified condition is true**. The condition is checked **before** the execution of the loop body.

Syntax:

```
while (condition) {
    // Code to execute as long as the condition is true
}
```

How it Works:

1. The condition is evaluated.
2. If the condition is true (non-zero), the code inside the loop body is executed.
3. After the body is executed, the condition is checked again.
4. The loop continues until the condition becomes false (zero).

Example: Printing numbers from 1 to 5:

```
#include <stdio.h>

int main() {
    int i = 1;

    while (i <= 5) {
        printf("%d\n", i);
        i++;  // Increment i
    }

    return 0;
}
```

Explanation:

- The loop starts with `i = 1`.
- The condition `i <= 5` is checked.
- As long as `i` is less than or equal to 5, the loop prints the value of `i` and increments it.
- Once `i` becomes 6, the condition becomes false, and the loop stops.

Output:

```
1
2
3
4
5
```

When to Use `while`:

- When you **don't know** in advance how many iterations are needed, but you want to keep looping until a specific condition is met.
- Useful for loops where the condition needs to be checked **before** any code is executed.

---

2. `do-while` Loop

The `do-while` loop is similar to the `while` loop, but with one key difference: the condition is checked **after** the code inside the loop is executed. This means that the loop is **guaranteed to execute at least once**, even if the condition is false from the beginning.

Syntax:

c

```c
do {
    // Code to execute at least once
} while (condition);
```

How it Works:

1. The code inside the loop is executed first.
2. After executing the code, the condition is checked.
3. If the condition is true (non-zero), the loop repeats. If it's false (zero), the loop stops.

Example: Printing numbers from 1 to 5:

c

```c
#include <stdio.h>

int main() {
    int i = 1;

    do {
        printf("%d\n", i);
        i++;  // Increment i
    } while (i <= 5);

    return 0;
}
```

Explanation:

- The loop first prints `i` and increments it.
- After printing, the condition `i <= 5` is checked.
- The loop continues to execute as long as the condition is true.
- In this case, it prints numbers from 1 to 5, just like the `while` loop.

Output:

```
1
2
3
4
5
```

When to Use `do-while`:

- When you want the loop body to execute **at least once**, regardless of the condition.
- Useful for scenarios like repeatedly asking for user input until a valid value is entered (where you want to prompt the user at least once before checking the condition).

Key Differences Between `while` and `do-while` Loops

Feature	`while` Loop	`do-while` Loop
Condition Check	Condition is checked **before** the loop body is executed.	Condition is checked **after** the loop body is executed.
First Execution	The loop **may not execute** if the condition is false initially.	The loop will **always execute at least once**.
Use Case	Use when the loop may or may not execute, depending on the condition.	Use when the loop should always run at least once.
Syntax	`while (condition) { /* code */ }`	`do { /* code */ } while (condition);`

Comparison with Example

Consider a situation where you need to check if a user enters a positive number. Using both types of loops will give slightly different behavior if the input is initially invalid.

Using `while` loop:

```
#include <stdio.h>

int main() {
    int num;

    printf("Enter a positive number: ");
    scanf("%d", &num);

    while (num <= 0) {
```

```
        printf("Invalid number. Enter a positive number: ");
        scanf("%d", &num);
    }

    printf("You entered: %d\n", num);

    return 0;
}
```

- The loop **only starts** if the user enters a non-positive number (because the condition is checked first).

Using `do-while` loop:

```
#include <stdio.h>

int main() {
    int num;

    do {
        printf("Enter a positive number: ");
        scanf("%d", &num);
    } while (num <= 0);

    printf("You entered: %d\n", num);

    return 0;
}
```

- The loop **executes at least once** and prompts the user even if the condition (positive number) is false initially.

Conclusion:

- **`while` loop** is ideal when you want to ensure that a block of code executes **only if a condition is true** from the start.
- **`do-while` loop** is useful when you need to ensure the code inside the loop **executes at least once**, even if the condition is initially false.

Switch-Case Statement in C

The `switch-case` statement in C is used to control the flow of a program by selecting one of many code blocks to execute. It provides a cleaner and more readable alternative to writing multiple `if-else-if` statements, especially when you are comparing a single variable or expression against a list of possible values.

Syntax:

```
switch (expression) {
    case value1:
        // Code to execute if expression == value1
        break;
    case value2:
        // Code to execute if expression == value2
        break;
    ...
    default:
        // Code to execute if none of the above cases are true
}
```

How it Works:

1. The **`expression`** inside the `switch` is evaluated.

2. The program compares the result of the expression with the values in each **`case`**.
3. If a match is found, the code block corresponding to that `case` is executed.
4. The **`break`** statement ensures that the control comes out of the `switch` block after executing the matching `case`.
5. If no case matches the expression, the **`default`** block is executed (optional).
6. If there is no `break` statement, the program will continue to execute the next case without checking its condition, leading to **fall-through behavior**.

Example: Simple Calculator Using `switch-case`

```
#include <stdio.h>

int main() {
    char operator;
    double num1, num2, result;

    // Asking user for input
    printf("Enter an operator (+, -, *, /): ");
    scanf("%c", &operator);

    printf("Enter two operands: ");
    scanf("%lf %lf", &num1, &num2);

    // Using switch-case to perform calculation based on the operator
    switch (operator) {
        case '+':
            result = num1 + num2;
            printf("Result: %.2lf + %.2lf = %.2lf\n", num1, num2,
result);
            break;
        case '-':
```

```
            result = num1 - num2;
            printf("Result: %.2lf - %.2lf = %.2lf\n", num1, num2,
result);
            break;
        case '*':
            result = num1 * num2;
            printf("Result: %.2lf * %.2lf = %.2lf\n", num1, num2,
result);
            break;
        case '/':
            if (num2 != 0) {
                result = num1 / num2;
                printf("Result: %.2lf / %.2lf = %.2lf\n", num1, num2,
result);
            } else {
                printf("Error! Division by zero.\n");
            }
            break;
        default:
            // If operator doesn't match any case
            printf("Invalid operator.\n");
    }

    return 0;
}
```

Explanation:

1. **Expression**: The variable `operator` is evaluated inside the `switch`.
2. **Case Matching**: The program checks if `operator` matches one of the cases (`+`, `-`, `*`, or `/`).

3. **Execution**: If a match is found (e.g., if the user enters `+`), the corresponding block of code is executed.
4. **Break**: The `break` statement ensures that after executing the matching case, the program exits the `switch` block and doesn't "fall through" to the next case.
5. **Default**: If the user enters an invalid operator (anything other than `+`, `-`, `*`, `/`), the `default` block executes, printing an error message.

Output:

arduino

```
Enter an operator (+, -, *, /): +
Enter two operands: 10 20
Result: 10.00 + 20.00 = 30.00
```

Breakdown of `switch-case` Components:

1. **`switch (expression)`**:
   - The expression inside the `switch` is evaluated once.
   - This can be a variable or any expression that results in an integer or a character value.
   - **Note**: Floating-point values are not allowed as cases in `switch`.
2. **`case value:`**:
   - Each `case` keyword is followed by a constant value, which can be an integer or character.
   - When the expression evaluates to this value, the corresponding block of code is executed.
3. **`break;`**:
   - The `break` statement is used to terminate a `case` and exit the `switch` block.
   - Without `break`, the program will continue to execute the following cases even if they don't match, leading to **fall-through** behavior.
4. **`default:`**:
   - The `default` block is optional but useful for handling unexpected values.
   - It executes if none of the `case` values match the expression.
   - It acts like an `else` statement in an `if-else` construct.

Fall-Through Behavior

Without the `break` statement, a `case` will "fall through" to the next one, even if the next `case` doesn't match. This can be useful in some scenarios but often leads to unintended behavior.

Example of Fall-Through:

```c
#include <stdio.h>

int main() {
    int number = 2;

    switch (number) {
        case 1:
            printf("One\n");
        case 2:
            printf("Two\n");
        case 3:
            printf("Three\n");
        default:
            printf("Default\n");
    }

    return 0;
}
```

Explanation:

- Since there are no `break` statements, after printing `"Two"`, the program falls through to `case 3` and `default`, printing `"Three"` and `"Default"` as well.

Output:

```
Two
Three
Default
```

Key Points to Remember:

- **`switch-case` vs `if-else`**:
  - `switch-case` is more efficient and readable when you are checking a single variable against multiple constant values.
  - `if-else` is more flexible when checking complex conditions (e.g., ranges, logical expressions).
- **Allowed values**:
  - `switch` can only handle **integer** or **character** values (e.g., `int`, `char`), not floating-point (`float`, `double`) or string values.
- **Efficiency**:
  - In certain scenarios, `switch-case` is more efficient than multiple `if-else` checks because compilers can optimize `switch` statements to use **jump tables**, making them faster for large sets of cases.

When to Use `switch-case`:

- **Menu-driven programs**: Where each menu option is mapped to a specific action.
- **State-based logic**: When you have a variable that represents a state (e.g., game levels or error codes).
- **Simple comparisons**: When you're comparing a single variable to multiple known values.

Conclusion:

The `switch-case` statement is an efficient and readable way to handle multiple possible values of a single variable. It's ideal for scenarios where you have discrete, constant values and want to avoid the clutter of multiple `if-else` statements. Always remember to use `break` to prevent fall-through unless fall-through behavior is desired.

For Loop in C

The **`for` loop** is one of the most commonly used loops in C. It is typically used when the number of iterations is known beforehand, as it allows you to control the initialization, condition, and update of the loop in a single line, making the code compact and easy to manage.

Syntax:

```
for (initialization; condition; update) {
```

```
    // Code to be executed repeatedly
}
```

Explanation:

1. **Initialization**: This is executed only once at the beginning of the loop. It is generally used to initialize a loop control variable (e.g., `int i = 0;`).
2. **Condition**: This is evaluated before each iteration. If the condition is **true**, the loop body is executed. If it's **false**, the loop terminates.
3. **Update**: This is executed after each iteration of the loop body. It is typically used to modify the loop control variable (e.g., `i++`).

Flow of a `for` Loop:

1. The loop starts by executing the **initialization** step (e.g., setting `i = 0`).
2. The **condition** is checked. If it evaluates to `true`, the loop body is executed.
3. After executing the loop body, the **update** step (e.g., `i++`) is executed.
4. The condition is checked again. If it's still true, the loop repeats.
5. This process continues until the condition becomes `false`, at which point the loop terminates.

Example: Print Numbers from 1 to 5:

```
#include <stdio.h>

int main() {
    int i;

    // Loop from 1 to 5
    for (i = 1; i <= 5; i++) {
        printf("%d\n", i);  // Print the current value of i
    }

    return 0;
```

```
}
```

Explanation:

1. **Initialization**: `i = 1` (initial value of `i` is set to 1).
2. **Condition**: `i <= 5` (the loop will continue as long as `i` is less than or equal to 5).
3. **Update**: `i++` (after each iteration, `i` is incremented by 1).
4. The loop will print the numbers 1 to 5, and then exit.

Output:

```
1
2
3
4
5
```

Parts of a `for` Loop:

1. **Initialization**:
   - This is where you initialize the loop control variable.
   - Example: `int i = 0;`
   - It runs **only once** at the beginning of the loop.
2. **Condition**:
   - This is the expression that is evaluated **before each iteration**.
   - The loop runs as long as the condition is **true**.
   - Example: `i <= 5`
   - If the condition is false, the loop stops.
3. **Update**:
   - This part is executed **after each iteration** of the loop.
   - It is typically used to update the loop control variable.
   - Example: `i++` (increments `i` by 1).
4. **Loop Body**:
   - This is the block of code that runs repeatedly as long as the condition is true.
   - Example: `printf("%d\n", i);` (prints the value of `i`).

Variations of `for` Loop:

1. Omitting Initialization:

You can declare and initialize the control variable outside the loop, making the initialization part empty.

```
#include <stdio.h>

int main() {
    int i = 1;  // Initialization outside the loop

    for (; i <= 5; i++) {  // No initialization inside for loop
        printf("%d\n", i);
    }

    return 0;
}
```

2. Infinite `for` Loop:

You can create an infinite loop by omitting the condition (or using `true` as the condition).

```
for (;;) {
    // Infinite loop, since there is no condition to terminate it
    printf("This will run forever!\n");
}
```

- This loop will continue until you manually break it (e.g., using a `break` statement or an external condition).

3. Using Multiple Variables:

You can also use multiple variables in the initialization and update parts of the loop.

```
for (int i = 0, j = 10; i < j; i++, j--) {
    printf("i = %d, j = %d\n", i, j);
}
```

Output:

```
i = 0, j = 10
i = 1, j = 9
i = 2, j = 8
i = 3, j = 7
i = 4, j = 6
```

For Loop vs. Other Loops (`while`, `do-while`)

Feature	For Loop	While Loop	Do-While Loop
Use Case	Best when the number of iterations is known.	Best when the number of iterations is unknown.	Best when the loop must run at least once.
Condition Check	Checks the condition before each iteration.	Checks the condition before each iteration.	Checks the condition **after** each iteration.
Structure	Combines initialization, condition, and update in one line.	Initialization, condition, and update are separate.	Similar to `while`, but the condition is at the end.
Guaranteed to Run?	No (may never run if the condition is false initially).	No (may never run if the condition is false initially).	Yes (always runs at least once, regardless of condition).

Conclusion:

The `for` loop is ideal when the number of iterations is known beforehand, as it allows you to organize the initialization, condition, and update steps in a single line. It is versatile and can be easily customized for different use cases, making it a staple in C programming.